



Lab Title: Bash Scripting Lab Exercise

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: Fundamentals of Networking

Instructor Name: Zakariyya Dahuru Musa

Date of Submission: 03/11/2025

Executive Summary

This lab was focused on learning and practicing Bash scripting on a Linux system. I worked through a series of 20 exercises that helped me understand how to automate tasks, work with files, check system information, and interact with users through the terminal. Each exercise built on the previous one, helping me improve my understanding of basic scripting logic, commands, and structure. By completing the lab, I learned how to write simple scripts that make daily Linux operations faster and easier.

Lab Objective

The main goal of this lab was to help me get comfortable writing and running Bash scripts. Specifically, the objectives were to:

- Learn how to use input and output in Bash.
- Understand how to use condition statements, loops, and variables.
- Practice automating simple system tasks like checking files, counting items, and showing system info.
- Gain experience working directly in the terminal.
- Develop problem-solving skills using real scripting examples.

Tools & Resources

Bash Shell: used to write and run all script

Linux Terminal: main working environment for executing commands and testing scripts.

Text Editor (nano & vim): for writing and editing scripts files.

Basic Linux Commands (ls, who, ping, du, free, date, etc.): used in different exercise to perform tasks and get system information.

File System: used for exercise involving files, directories, and permission.

Methodology

I carried out each exercise step by step using the Linux terminal.

1. I started by reading each task and planning what commands would be needed.
2. I used a text editor (like nano or vim) to write the script.
3. I saved the file with a .sh extension and made it executable using `chmod +x`.
4. I ran the script in the terminal to test if it worked correctly.
5. When errors came up, I went back to fix them and re-ran the script until I got the expected output.
6. I also added comments in each script to explain what each part does.

This hands-on approach helped me understand both the logic and the syntax of Bash scripting better.

Lab Exercise 1

In this lab exercise, I created a simple Bash script named `greetuser.sh` to prompt the user for their first and last names. The script uses the `read` command to capture input from the user and stores it in the variables `firstname` and `lastname`.

The `echo` command is then used to display a personalized greeting by combining the values stored in these variables.

Before running a Bash script, it must have execute permissions. This is done using the command: `chmod u+x greetuser.sh`

When executed, the script interacts with the user and outputs the greeting: **Hello Ifeanyichukwu Raphael**

This exercise demonstrates how to capture user input and use variables to generate dynamic output in a Bash script.

```
raph@kali: ~  
#!/bin/bash  
  
#Prompt user for their name  
read -p "Enter your first name: " firstname  
read -p "Enter your last name: " lastname  
  
#Output user name  
echo Hello $firstname $lastname
```

```
(raph@kali)-[~]  
$ vim greetuser.sh  
  
(raph@kali)-[~]  
$ chmod u+x greetuser.sh  
  
(raph@kali)-[~]  
$ ./greetuser.sh  
Enter your first name: Ifeanyichukwu  
Enter your last name: Raphael  
Hello Ifeanyichukwu Raphael  
  
(raph@kali)-[~]  
$
```

Lab Exercise 2

In this lab exercise, I created a Bash script [checkfile.sh](#) that checks whether a specified file exists in the current directory. The script first prompts the user to enter a filename and stores it in a variable. It then uses a conditional statement with the `-f` option to determine if the file exists. Depending on the check, the script outputs a message indicating whether the file is present or not.

This exercise demonstrates the use of:

- User input handling with `read`
- Variables to store input
- Conditional statements (`if-else`) for logic control

- File testing (**-f**) in Bash
- nano **checkfile.sh** opens the script in the nano editor for editing.

It reinforces basic Bash scripting skills and introduces simple file management checks that are commonly used in automation and system administration tasks.

```

+ raph@kali: ~
GNU nano 8.6 checkfile.sh
#!/bin/bash

#Prompt the user to enter a filename
read -p "Enter a filename: " filename

#Check if the file exist in the current directory
if [ -f $filename ]; then
    echo "The file $filename exists in the current directory."
else
    echo "The file $filename does NOT exist in the current diectory."
fi

```

```

(raph@kali)-[~]
$ nano checkfile.sh

(raph@kali)-[~]
$ chmod u+x checkfile.sh

(raph@kali)-[~]
$ ./checkfile.sh
Enter a filename: mme.txt
The file mme.txt does NOT exist in the current diectory.

(raph@kali)-[~]
$

```

Lab Exercise 3

This lab demonstrates how to use Bash scripting [datetime.sh](#) to display the current date and time.

- `vim datetime.sh` - Opens the script in the Vim editor.
- `./datetime.sh` - Runs the script.
- `date +%Y-%m-%d` - Retrieves the current date in YYYY-MM-DD format and stores it in the variable `current_date`.
- `date +%H:%M:%S` - Retrieves the current time in HH:MM:SS format and stores it in the variable `current_time`.
- `echo` - Outputs the combined date and time in a readable format:
- “The current date and time is: **YYYY-MM-DD HH:MM:SS**”, 2025-11-01 08:05:59

Key Concepts Covered:

- Using date command with format specifiers.
- Assigning command output to variables using `$(...)`.
- Displaying variable values with `echo`.

```
#!/bin/bash

current_date=$(date +%Y-%m-%d)
current_time=$(date +%H:%M:%S)
echo "The current date and time is: $current_date $current_time"
```

```
(raph@kali)-[~]
$ vim datetime.sh

(raph@kali)-[~]
$ ./datetime.sh
The current date and time is: 2025-11-01 08:05:59

(raph@kali)-[~]
$
```

Lab Exercise 4

In this lab exercise, I enhanced my skills by building a simple calculator that accepts two numbers from the user and allows them to choose an arithmetic operation: addition, subtraction, multiplication, or division. I used **if - elif - else** statements to perform the chosen operation and display the result. Special attention was given to handle division by zero to avoid errors. This exercise taught me how to implement conditional logic, perform arithmetic calculations using `$((` `))`, and handle user choices effectively.

Overall, these exercises improved my understanding of Bash scripting basics such as user input handling, conditional statements, arithmetic operations, and output formatting. I am now more confident in writing scripts that interact with users and perform calculations dynamically.

```
#!/bin/bash

#Prompt user to enter two number
read -p "Enter your first number: " num1
read -p "Enter your second number: " num2

#Prompt user to enter operation
echo "Choose an operation:"
echo "1.Addition (+)"
echo "2.Subtraction (-)"
echo "3.Multiplication (*)"
echo "4.Division (/)"

read -p "Enter your choice (1-4): " choice

# Perform operation using if-else statements
if [ $choice -eq 1 ]; then
    result=$((num1 + num2))
    echo "Result: $num1 + $num2 = $result"

elif [ $choice -eq 2 ]; then
    result=$((num1 - num2))
    echo "Result: $num1 - $num2 = $result"

elif [ $choice -eq 3 ]; then
    result=$((num1 * num2))
    echo "Result: $num1 * $num2 = $result"

elif [ $choice -eq 4 ]; then
    if [ $num1 -eq 0 ]; then
        echo "Error: Division by zero is not allowed."
    else
        result=$((num1 / num2))
        echo "Result: $num1 / $num2 = $result"
    fi
else
    -- INSERT --
fi
```

```
(raph@kali)-[~]  
$ vim simplecalculator.sh
```

```
(raph@kali)-[~]  
$ ./simplecalculator.sh  
Enter your first number: 4  
Enter ypur second number: 2  
Choose an operation:  
1.Addition (+)  
2.Subtraction (-)  
3.Multiplication (*)  
4.Division (/)  
Enter your choice (1-4): 1  
Result: 4 + 2 = 6
```

```
(raph@kali)-[~]  
$ ./simplecalculator.sh  
Enter your first number: 6  
Enter ypur second number: 4  
Choose an operation:  
1.Addition (+)  
2.Subtraction (-)  
3.Multiplication (*)  
4.Division (/)  
Enter your choice (1-4): 2  
Result: 6 - 4 = 2
```

```
(raph@kali)-[~]  
$ ./simplecalculator.sh  
Enter your first number: 4  
Enter ypur second number: 2  
Choose an operation:  
1.Addition (+)  
2.Subtraction (-)  
3.Multiplication (*)  
4.Division (/)  
Enter your choice (1-4): 3
```

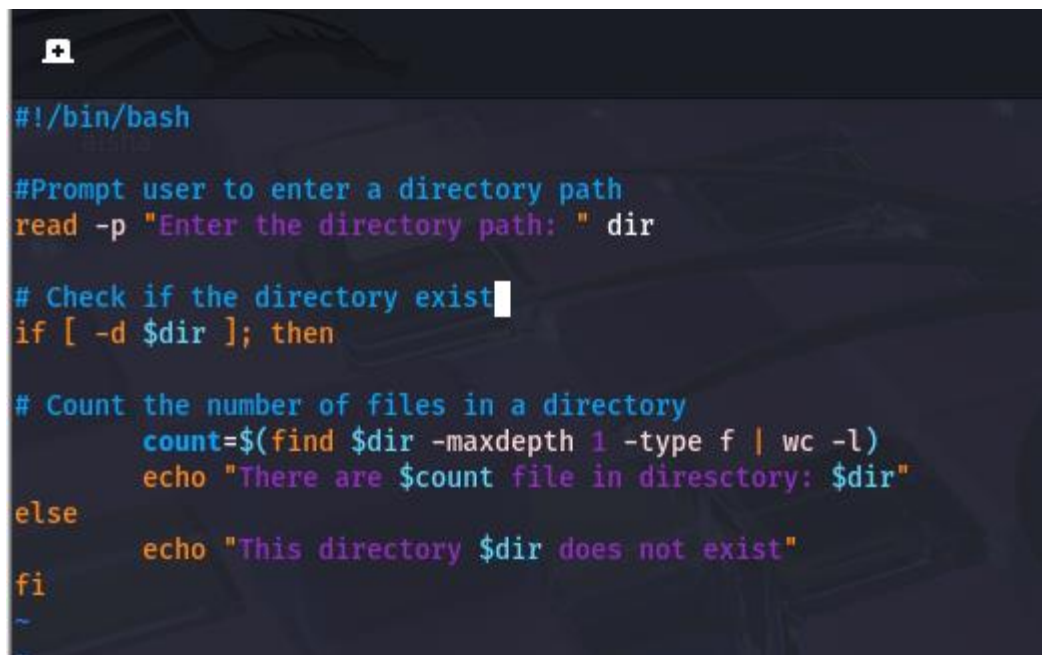
Lab Exercise 5

In this lab exercise, I worked on two Bash scripts focused on directory handling and file counting. I wrote a script that prompts the user to enter a directory path. The script then checks if

the specified directory exists using the **-d** test condition. If the directory exists, the script confirms this to the user; otherwise, it informs the user that the directory does not exist. This exercise helped me practice conditional statements and user input validation in Bash scripting.

Building on the first script, I enhanced it to count the number of files inside the given directory. Using the find command with options to limit the search to the top-level files and **wc -l** to count them, the script outputs the total number of files found. This taught me how to combine commands and capture their output in variables to present useful information to the user.

These exercises strengthened my skills in interacting with the file system through Bash scripts, including checking directory existence and working with command output. They also reinforced my understanding of input handling, conditionals, and command substitution in shell scripting.

A screenshot of a terminal window with a dark background. The terminal shows a Bash script being executed. The script starts with a shebang line, followed by a comment and a prompt for a directory path. It then uses a read command to capture input. A conditional statement checks if the directory exists using the -d test. If it exists, it uses find and wc -l to count files and echoes the result. If not, it echoes a message that the directory does not exist. The script ends with a fi statement.

```
#!/bin/bash

#Prompt user to enter a directory path
read -p "Enter the directory path: " dir

# Check if the directory exist
if [ -d $dir ]; then

# Count the number of files in a directory
    count=$(find $dir -maxdepth 1 -type f | wc -l)
    echo "There are $count file in diredctory: $dir"
else
    echo "This directory $dir does not exist"
fi
```

```
(raph@kali)-[~]
$ vim countfile_dir.sh

(raph@kali)-[~]
$ chmod countfile_dir.sh
chmod: missing operand after 'countfile_dir.sh'
Try 'chmod --help' for more information.

(raph@kali)-[~]
$ chmod u+x countfile_dir.sh

(raph@kali)-[~]
$ ./countfile_dir.sh
Enter the directory path: Downloads
There are 1 file in diredtory: Downloads

(raph@kali)-[~]
$
```

Lab Exercise 6

In this lab exercise, I created a Bash script named `check_loginuser.sh` that prompts the user to enter a username and checks whether that user is currently logged into the system. The script uses the `who` command to list active sessions and `grep` with the `-wq` options to search for an exact match of the entered username. If the user is found in the list, the script confirms that the user is logged in; otherwise, it reports that the user is not. This exercise helped me practice user input handling, conditional logic, and the use of system commands to interact with session data in Bash scripting.

```
#!/bin/bash

# Prompt user to enter username
read -p "Enter your username: " user

# Check the current login user
if who | grep -wq "$user"; then
    echo "User '$user' is currently logged in."
else
    echo "User '$user' is not logged in."
fi
```

```
(raph@kali) [~]
$ vim check_loginuser.sh

(raph@kali)-[~]
$ ./check_loginuser.sh

Enter your username: raph
User 'raph' is currently logged in.

(raph@kali)-[~]
```

Lab Exercise 7

In this lab exercise, I created a Bash script named [numbercamp.sh](#) to compare two numbers entered by the user. The script prompts the user for two numbers and then uses conditional statements (**if**, **elif**, **else**) to determine whether the first number is greater than, less than, or equal to the second number.

Initially, I encountered a syntax error (**unexpected end of file from 'if'**) due to a missing **fi** to close the **if** statement. After correcting this, the script ran successfully and displayed the correct output for different inputs:

- When the first number is greater than the second, it prints that it is greater.
- When the first number is smaller, it prints that it is less.
- When the numbers are equal, it prints that they are equal.

This exercise helped me practice handling user input, implementing conditional logic, and debugging common syntax errors in Bash scripts. It strengthened my understanding of flow control using **if** – **elif** – **else** structures in shell scripting.



raph@kali: ~

```
#!/bin/bash

# Prompt a user to enter two numbers
read -p "Enter your first number: " num1
read -p "Enter your second number: " num2

# Comparing the two numbers
if [ $num1 -gt $num2 ]; then
    echo "$num1 is greater than $num2."
elif [ $num1 -eq $num2 ]; then
    echo "$num1 is equal to $num2."
else
    echo "$num1 is less than $num2."
fi

~
~
```

```
└─$ chmod u+x numbercamp.sh

└─(raph@kali)-[~]
└─$ ./numbercamp.sh
Enter your first number: 1
Enter your second number: 3
./numbercamp.sh: line 13: syntax error: unexpected end of file from `if' command
on line 8

└─(raph@kali)-[~]
└─$ vim numbercamp.sh

└─(raph@kali)-[~]
└─$ ./numbercamp.sh
Enter your first number: 2
Enter your second number: 1
2 is greater than 1.

└─(raph@kali)-[~]
└─$ ./numbercamp.sh
Enter your first number: 50
Enter your second number: 100
50 is less than 100.

└─(raph@kali)-[~]
└─$ ./numbercamp.sh
Enter your first number: 10
Enter your second number: 10
10 is equal to 10.
```

Lab Exercise 8

In this lab exercise, I created and executed a Bash script on Kali Linux that counts from 1 up to a number entered by the user. I wrote the script using `vim`, starting with a shebang `#!/bin/bash` and using the `read` command to prompt for input. I implemented a `while` loop that printed each count and ended with a "Loop completed" message.

After saving the script, I made it executable with `chmod u+x` and ran it successfully. Entering the number 10 produced output from "Loop 1" to "Loop 10." This exercise helped me understand user input handling, loop control, and script execution in Bash. It was a great hands-on experience that improved my confidence in shell scripting.

```
$ vim numbercamp.sh

(raph@kali)-[~]
$ vim loopnumbers.sh

(raph@kali)-[~]
$ chmod u+x loopnumbers.sh

(raph@kali)-[~]
$ ./loopnumbers.sh
Enter your first number: 10
Loop 1.
Loop 2.
Loop 3.
Loop 4.
Loop 5.
Loop 6.
Loop 7.
Loop 8.
Loop 9.
Loop 10.
Loop completed.

(raph@kali)-[~]
$
```

Lab Exercise 9

In this lab, I created Bash scripts `systeminfo.sh` used command substitution to capture system details like `hostname`, `kernel version`, and `uptime`, then displayed them using `echo`.

I edited both scripts using `vim`, made them executable with `chmod u+x`, and ran them successfully from the terminal. These exercises helped me understand user input handling, loop control, variable assignment, and system command integration in Bash. They were a great step forward in building my confidence with Linux scripting.

```
#!/bin/bash

# Assigning commands to a variable
host_name=$(hostname)
kernel_version=$(uname -r)
show_uptime=$(uptime)

# Outputting commands
echo "Hostname: $host_name."
echo "Kernel version: $kernel_version."
echo "Uptime: $show_uptime."
~
~
~
```

```
(raph@kali)-[~]
$ vim systeminfo.sh

(raph@kali)-[~]
$ chmod u+x systeminfo.sh

(raph@kali)-[~]
$ ./systeminfo.sh
Hostname: kali.
Kernel version: 6.16.8+kali-amd64.
Uptime: 17:10:34 up 11:35, 1 user, load average: 0.13, 0.17, 0.17.
```

Lab Exercise 10

The goal of this lab was to create a Bash script that allows a user to back up any file by automatically generating a copy with a `.bak` extension.

I wrote a Bash script named `backupfile.sh`. The script prompts the user to enter the name of a file, checks whether that file exists in the current directory, and if it does, creates a backup using the `cp` command. The new backup file is named with the original filename followed by the `.bak` extension. If the file does not exist, the script displays an error message.

After running the script and entering the filename `myscript.sh`, the backup file `myscript.sh.bak` was successfully created in the same directory. The output confirmed the success of the operation.

This lab helped me understand how to use basic Bash scripting concepts such as user input `read`, conditional statements `if - else`, and file manipulation commands (`cp`). The script effectively demonstrates how automation can simplify repetitive tasks like creating backups in a Linux environment.

```
#!/bin/bash

# Prompt the user for a filename
read -p "Enter your filename: " filename

# Script to backup a file with .bak extention
if [ -f "$filename" ]; then
    cp "$filename" "$filename.bak"
    echo "Backup created successfully: $filename.bak"
else
    echo "File '$filename' not found!"
fi
```

```
(raph@kali)-[~]
$ vim backupfile.sh

(raph@kali)-[~]
$ ./backupfile.sh
Enter your filename: myscript.sh
Backup created successfully: myscript.sh.bak

(raph@kali)-[~]
$ ls -l myscript.sh*
-rw-rw-r-- 1 raph raph 0 Nov  2 06:27 myscript.sh
-rw-rw-r-- 1 raph raph 0 Nov  2 06:34 myscript.sh.bak
```

Lab Exercise 11

In this lab session, I developed and executed a Bash script named `checkinternet.sh` to test network connectivity by verifying the reachability of a given host. I began by writing the script using the `vim` editor on my Kali Linux system, incorporating basic shell scripting techniques

such as user input handling, conditional logic, and command execution. The script prompts the user to enter a hostname or IP address, then uses the `ping` command to check if the host is reachable. When I tested it with the loopback address `127.0.0.1`, the script successfully confirmed connectivity by displaying "Host 127.0.0.1 is reachable!" This exercise helped reinforce my understanding of Bash scripting fundamentals and demonstrated how simple automation can assist in network diagnostics.

```
#!/bin/bash

# Prompting user for host
read -p "Enter hostname or IP: " host

# pinging the host and output the result
if ping -c 1 "$host" >/dev/null; then
    echo "Host \"$host\" is reachable!"
else
    echo "Host \"$host\" is not reachable."
fi

~
~
```

```
(raph@kali)-[~]
$ vim checkinternet.sh

(raph@kali)-[~]
$ ./checkinternet.sh
Enter hostname or IP: 127.0.0.1
Host 127.0.0.1 is reachable!
```

Lab Exercise 12

In this lab session, I explored the fundamentals of shell scripting on a Kali Linux system by creating a Bash script named `diskusage.sh`. Using `Vim`, I wrote a script that calculates and displays the disk usage of the current directory with the `du -sh` command, making the output human-readable. After saving the script, I granted it execution permissions using `chmod u+x`, reinforcing my understanding of Unix file permission management. Running the script produced a clear output "66M" indicating the total space used, which gave me immediate insight into my

directory's storage footprint. This hands-on exercise not only sharpened my scripting skills but also deepened my confidence in navigating and automating tasks within the Linux environment.

```
raph@kali: ~  
#!/bin/bash  
echo "Calculating disk usage for the current directory."  
# Displays the disk usage  
du -sh  
~  
~  
~
```

```
(raph@kali)-[~]  
$ vim diskusage.sh  
  
(raph@kali)-[~]  
$ chmod u+x diskusage.sh  
  
(raph@kali)-[~]  
$ ./diskusage.sh  
Calculating disk usage for the current directory.  
66M .
```

Lab Exercise 13

In this lab exercise, I practiced writing and executing a Bash script that prints all even numbers from 1 up to a user-defined limit. I began by creating the script using **Vim**, then made it executable with the **chmod +x** command. The script prompts the user to enter a number, stores it in a variable, and uses the **seq** command to generate even numbers in steps of 2. When I ran the script and entered the number 20, it correctly displayed all even numbers from 2 to 20. This task helped reinforce my understanding of user input handling, basic control flow, and command-line utilities in a Unix-like environment.

```
#!/bin/bash

read -p "Enter a number: " N

echo "Even numbers from 1 to $N:"

seq 2 2 $N

~
~
~
~
~
```

```
(raph@kali)-[~]
└─$ vim even_number.sh

(raph@kali)-[~]
└─$ chmod +x even_number.sh

(raph@kali)-[~]
└─$ ./even_number.sh
Enter a number: 20
Even numbers from 1 to 20:
2
4
6
8
10
12
14
16
18
20
```

Lab Exercise 14

In this lab exercise, I practiced writing and executing a Bash script that checks the permissions of a user-specified file. I started by creating the script using **vim** and made it executable with the **chmod +x** command. The script prompts the user to enter a filename, stores it in a variable, and uses conditional statements with **-r**, **-w**, and **-x** flags to determine if the file is readable, writable, and executable. When I ran the script and entered "**systeminfo.sh**", it correctly reported that the file is readable and executable but not writable. This task helped reinforce my understanding of file permission checks, user input handling, and basic scripting logic in a Unix-like environment.

```
#!/bin/bash

# Prompt the user to enter a filename
read -p "Enter a filename: " file

# Check if file is readable
if [ -r "$file" ]; then
    echo "The file is readable."
else
    echo "The file is not readable."
fi

# Check if file is writable
if [ -w "$File" ]; then
    echo "The file is writable."
else
    echo "The file is not writable."
fi

# Check if file is executable
if [ -x "$file" ]; then
    echo "The file is executable."
else
    echo "The file is not executable."
fi
```

```
(raph@kali)-[~]
$ ./checkfile_perm.sh
Enter a filename: systeminfo.sh
The file is readable.
The file is not writable.
The file is executable.

(raph@kali)-[~]
$
```

Lab Exercise 15

In this lab exercise, I practiced writing and executing a Bash script that generates a memory usage report on a Unix-like system. I began by creating the script using `vim`, naming it `monitor_memoryusage.sh`. The script starts with a shebang `#!/bin/bash` to specify the Bash shell, followed by an `echo` statement to display a header message. It then uses the `free -h`

command to output memory statistics in a human-readable format, and ends with another `echo` statement confirming successful report generation. After saving the script, I made it executable using the `chmod +x` command. When I ran the script, it displayed a detailed breakdown of memory and swap usage, including total, used, free, and available memory. This task helped reinforce my understanding of shell scripting basics, file permissions, and system monitoring tools commonly used in Linux environments.

```
#!/bin/bash
echo "Memory usage report."
free -h
echo "Report generated successfully."
```

```
(raph@kali)-[~]
$ vim monitor_memoryusage.sh

(raph@kali)-[~]
$ chmod +x monitor_memoryusage.sh

(raph@kali)-[~]
$ ./monitor_memoryusage.sh
Memory usage report.

```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	1.6Gi	1.3Gi	6.8Mi	1.1Gi	2.1Gi
Swap:	1.6Gi	0B	1.6Gi			

```
Report generated successfully.
```

Lab Exercise 16

In this lab exercise, I practiced writing and executing a Bash script that displays the number of users currently logged into the system. I began by creating the script using `vim` and saved it as `displaylogin_user.sh`. The script starts with a shebang `#!/bin/bash` to specify the Bash shell, followed by an `echo` statement to print a message. It then uses a pipeline of commands- `who | grep ^ | wc -l`, to count the number of logged-in users by listing active sessions, filtering non-empty lines, and counting them. After saving the script, I made it executable using `chmod +x`, and then ran it from the terminal. The output correctly displayed the message and the

number of logged-in users. This task helped reinforce my understanding of user session monitoring, command chaining, and basic shell scripting techniques in a Unix-like environment.

```
#!/bin/bash  
  
echo "Current logged in user."  
  
who | grep ^ | wc -l
```

```
(raph@kali)-[~]  
$ vim displaylogin_user.sh  
  
(raph@kali)-[~]  
$ chmod +x displaylogin_user.sh  
  
(raph@kali)-[~]  
$ ./displaylogin_user.sh  
Current logged in user.  
2
```

Lab Exercise 17

In this lab exercise, I practiced writing and executing a Bash script that implements a countdown timer. I began by creating the script using `vim` and saved it as `countdown_timer.sh`. The script starts with a shebang `#!/bin/bash` to specify the Bash shell, followed by a `read` command that prompts the user to enter a number of seconds. It then prints a starting message and enters a `while` loop that decrements the time variable each second using `sleep` and `((time--))`, displaying the countdown in real time. After saving the script, I made it executable using `chmod +x`, and then ran it from the terminal. The output correctly displayed each second until it reached zero, followed by the message "Time's up!" This task helped reinforce my understanding of user input handling, loop control, and basic shell scripting in a Unix-like environment.

```
#!/bin/bash
# Prompt user to enter a number (seconds)
read -p "Enter a number of seconds for countdown: " time

echo "Starting countdown from $time..."

while [ "$time" -gt 0 ];
do
    echo "$time"
    sleep 1
    ((time--))
done

echo "Time's up!"
```

```
(raph@kali)-[~]
$ vim countdown_timer.sh

(raph@kali)-[~]
$ ./countdown_timer.sh
Enter a number of seconds for countdown: 20
Starting countdown from 20....
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
Time's up!
```

Lab Exercise 18

In this lab exercise, I created and executed a Bash script that counts the number of lines in a specified file. Using **vim**, I wrote the script and saved it as **fileline_count.sh**. The script begins with a shebang **#!/bin/bash** and uses the **read** command to prompt the user for a filename. It then checks if the file exists using an **if** statement with the **-f** flag. If the file is found, the script uses **wc -l** to count the lines and displays the result. If the file does not exist, it prints an error message. After making the script executable with **chmod +x**, I ran it from the terminal, successfully verifying its functionality with a test file. This exercise strengthened my understanding of file handling, conditional logic, and command-line utilities in shell scripting.

```
#!/bin/bash

# Prompt user to enter a filename
read -p "Enter a filename: " file

# Check if the file exist
if [ -f "$file" ]; then

# Counting the number of lines
    line_count=$(wc -l < "$file")
    echo "The file '$file' has '$line_count' lines."
else
    echo "File '$file' does not exist."
fi
```

```
(raph@kali)-[~]
$ vim fileline_count.sh

(raph@kali)-[~]
$ ./fileline_count.sh
Enter a filename: systeminfo.sh
The file 'systeminfo.sh' has '11' lines.

(raph@kali)-[~]
$
```

Lab Exercise 19

In this lab exercise, I developed a Bash script named `simplepassword-check.sh` that performs basic password validation. The script begins by defining a fixed password value stored in a variable. It then prompts the user to enter a password using the `read` command with the `-sp` flag, which ensures the input remains hidden for privacy. After capturing the user's input, the script compares it to the predefined password using a conditional `if` statement. If the entered password matches, the script prints "Access granted"; otherwise, it displays "Access denied." I tested the script by running it with both incorrect and correct passwords to verify its behavior. This exercise helped me understand secure input handling, string comparison, and conditional logic in Bash scripting, reinforcing how simple authentication mechanisms can be implemented in shell scripts.

```
#!/bin/bash

# Setting a predefined password
password="icdfa123"

# Prompt the user for a password
read -sp "Enter your password: " user_input
echo

# Check if the entered password matches
if [ "$user_input" == "$password" ]; then
    echo "Access granted"
else
    echo "Access denied"
fi
~
```

```
(raph@kali)-[~]
└─$ vim simplepassword-check.sh

(raph@kali)-[~]
└─$ chmod +x simplepassword-check.sh

(raph@kali)-[~]
└─$ ./simplepassword-check.sh
Enter your password:
Access denied

(raph@kali)-[~]
└─$ ./simplepassword-check.sh
Enter your password:
Access granted
```

Lab Exercise 20

In this lab exercise, I practiced writing and executing a Bash script that creates a directory structure based on user input. I began by opening **vim** and writing the script, then saved it as `create_dir_structure.sh`. The script starts with a shebang `#!/bin/bash` to specify the Bash shell, followed by a `read` command that prompts the user to enter the name of the main directory. It then uses `mkdir -p` to create the main directory and three subdirectories named

`docs`, `images`, and `scripts` inside it. After saving the script, I made it executable using `chmod +x`, and ran it from the terminal. The script successfully created the directory structure and displayed a confirmation message. I verified the result by navigating into the new directory and listing its contents recursively with `ls -R`, which showed the three empty subdirectories. This exercise helped reinforce my understanding of user input handling, directory creation, and basic shell scripting in a Unix-like environment.

```
#!/bin/bash

# Prompt user for the main directory name
read -p "Enter the main directory name: " dir

# Create the main directory
mkdir -p "$dir"

# Create the subdirectories inside it
mkdir -p "$dir/docs" "$dir/images" "$dir/scripts"

# Display confirmation message
echo "Directory structure created successfully inside '$dir'."
~
~
~
~
~
```

```
(raph@kali) [~]
$ vim create_dir_structure.sh

(raph@kali) [~]
$ chmod +x create_dir_structure.sh

(raph@kali) [~]
$ ./create_dir_structure.sh
Enter the main directory name: ICDFA
Directory structure created successfully inside 'ICDFA'.

(raph@kali) [~]
$ cd ICDFA

(raph@kali) [~/ICDFA]
$ ls -R
.:
docs images scripts

./docs:

./images:

./scripts:

(raph@kali) [~/ICDFA]
$
```

Conclusion

This collection of Bash scripting exercises showcases my ability to design practical, efficient scripts for a variety of system tasks. From user interaction and file handling to system monitoring and automation, each script reflects a hands-on understanding of core shell scripting principles. I've demonstrated proficiency in using conditionals, loops, command substitution, and integrating system utilities like **who**, **du**, **wc**, and **ping**. These projects highlight not only my technical skills but also my problem-solving approach-building scripts that are clear, purposeful, and user-friendly. Whether checking system status, validating input, or automating routine tasks, I've built a foundation that prepares me to contribute meaningfully to real-world scripting and system administration challenges. This lab exercise reflects my growth and readiness to take on more advanced scripting responsibilities in professional environments.

Reference

- GNU Bash Manual. (n.d.). *Bash Reference Manual*. Retrieved from <https://www.gnu.org/software/bash/manual/>

- ICDFEA Training Resources - *Introduction to Bash Scripting and File Automation*, 2025.
- Linux Manual Pages - `cp(1)`, `read(1)`, `test(1)` (Accessed via terminal using `man` command)
- YouTube Tutorials - *Bash Scripting Basics for Beginners*, various channels (Accessed in 2025).